Infosys DevOps Interview

Structured Answer Guide — Sanket Chopade

Q1. How would you approach migrating a monolithic application to microservices? What steps would you follow, and what key challenges might you encounter?

■ Migration Approach (Strangler Fig Pattern)

- **Assess & Decompose:** Map the monolith's modules, identify domain boundaries using Domain-Driven Design (DDD). Rank modules by change frequency and independent scalability need.
- **Define Service Contracts:** Design REST or gRPC APIs for each candidate service before writing a single line of code.
- **Containerise First:** Dockerise the entire monolith as a baseline — this forces you to resolve port, config, and dependency issues early.
- **Extract Incrementally:** Peel one service at a time (Strangler Fig). Route specific traffic to the new service via an API Gateway while the monolith handles the rest.
- **Set Up CI/CD Per Service:** Each microservice gets its own pipeline (Jenkins/GitHub Actions → ECR → EKS) with independent test gates.
- **Data Decoupling:** Move from a shared DB to per-service databases. Use event-driven patterns (Kafka/SQS) for cross-service data sync.
- **Observability from Day 1:** Instrument each service with Prometheus metrics, structured logs, and distributed tracing (Jaeger/X-Ray) before going live.

■ Key Challenges

- **Distributed data consistency:** ACID transactions no longer work across service boundaries — you must embrace eventual consistency or Sagas.
- **Network latency & failure handling:** In-process calls become HTTP/gRPC calls that can timeout — implement retries, circuit breakers (Resilience4j).
- **Service discovery & routing:** Kubernetes Service + Ingress or a Service Mesh (Istio) is needed; DNS-based discovery adds operational overhead.
- **Deployment complexity:** Multiple pipelines, versioning, and backward-compatible API changes demand strong DevOps maturity.
- **Testing:** Unit tests are insufficient — contract testing (Pact) and integration tests across services become critical.

Q2. After deploying an application, it becomes slow. How would you troubleshoot the issue and rectify it?

■ Structured Troubleshooting — Layer by Layer

- **Reproduce & Baseline:** Confirm slowness is real with load tests (k6/Locust). Capture p50/p95/p99 latency before touching anything.
- **Check Recent Changes:** Compare the deployment diff. Did resource limits, replicas, or config maps change? Roll back first if unsure.

• **Kubernetes Health Check:** *kubectl top pods / nodes* to spot CPU/memory saturation. Check HPA limits, pending pods, and OOMKill events.

• **Application Metrics (Prometheus + Grafana):** Look at request rate, error rate, and latency (RED method). Identify which endpoint is slow.

• **Infrastructure Layer:** Check RDS slow query logs, ElastiCache hit rate, S3 latency, and ALB target response times.

• **Network & DNS:** Verify inter-pod network policies, check CoreDNS pod health, and confirm ALB health check targets are all healthy.

• **Log Analysis:** Use CloudWatch Logs Insights or Loki to find timeout errors, DB connection pool exhaustion, or GC pauses.

■ Common Root Causes & Fixes

• **DB N+1 queries:** Add indexes, enable query caching, or introduce read replicas.

• **Pod resource starvation:** Tune CPU/memory requests and limits; adjust HPA thresholds.

• **Missing CDN caching:** Route static assets through CloudFront instead of hitting the origin.

• **Synchronous external calls:** Move to async queues (SQS/SNS) to avoid blocking request threads.

• **Connection pool exhaustion:** Increase DB max connections or add PgBouncer for connection pooling.

Q3. What happens if the Terraform state file in Amazon S3 is accidentally deleted? How would you recover it and prevent such incidents in future?

■ What Happens Immediately

• Terraform loses all awareness of existing infrastructure — it treats the next *terraform plan* as if resources don't exist.

• Running *terraform apply* would attempt to CREATE duplicate resources, causing conflicts or outright failures.

• No state = no safe way to destroy or update resources through Terraform.

■ Recovery Steps

• **Step 1 — Enable S3 Versioning (if done beforehand):** Restore the deleted object from a previous version via AWS Console → S3 → bucket → Versions tab.

• **Step 2 — If no versioning existed:** Manually import every resource back into a blank state using *terraform import*.

• **Step 3 — Validate:** After restore, run *terraform plan* — output should show **No changes** if state matches live infra.

• **Step 4 — Audit:** Check CloudTrail logs for who deleted the file and when. This is a security incident, not just ops.

■ Prevention (Best Practices)

• **Enable S3 Versioning** on the state bucket — makes every delete a soft delete recoverable instantly.

• **Enable MFA Delete** on the S3 bucket — prevents deletion without MFA, even by admins.

• **S3 Object Lock (WORM)** — configure a retention period to block deletions entirely for critical state files.

• **Restrict IAM permissions** — only the CI/CD role should have *s3:DeleteObject* on the state bucket; deny for humans.

• **DynamoDB state locking** — prevents concurrent *apply* runs that could corrupt state.

• **Automated state backups** — replicate state bucket to a secondary region using S3 Cross-Region Replication.

■ Terraform Backend Config (Reference)

```
terraform {
  backend "s3" {
    bucket         = "my-tf-state"
    key            = "prod/terraform.tfstate"
    region         = "ap-south-1"
    encrypt        = true
    dynamodb_table = "terraform-lock"
  }
}
```

Q4. What Jenkins strategy and pipeline type did you use?

■ Pipeline Type: Declarative Pipeline (Jenkinsfile in SCM)

• Used **Declarative Pipeline** syntax — more structured, readable, and GitOps-friendly than Scripted Pipelines.

• The Jenkinsfile lived in the application repo root — version-controlled alongside the code.

• Triggered automatically on every push via GitHub Webhooks.

■ Strategy: Multi-Stage with Environment Gates

• **Stages:** Checkout → Code Quality (SonarQube) → Container Scan (Trivy) → Docker Build → Push to ECR → Deploy to Dev EKS → Manual Gate → Promote to Staging.

• **Parallel stages** for unit tests and lint checks to reduce build time.

• **Environment-specific parameters** passed as Jenkins credentials and environment variables (no secrets in Jenkinsfile).

• Failed SonarQube quality gate or Trivy HIGH/CRITICAL findings **break the pipeline** — no deployment proceeds.

■ Sample Jenkinsfile Structure

```
pipeline {
  agent any
  stages {
    stage("Code Quality") { steps { sh "sonar-scanner ..." } }
    stage("Container Scan") { steps { sh "trivy image ..." } }
    stage("Build & Push") { steps { sh "docker build & push to ECR" } }
    stage("Deploy") { steps { sh "helm upgrade --install ..." } }
  }
}
```

Q5. What Kubernetes deployment strategy did you use in your project?

■ Strategy Used: Rolling Update (default Kubernetes strategy)

• Kubernetes replaces pods incrementally — new pods start before old ones terminate.

• Configured **maxSurge: 1** and **maxUnavailable: 0** to ensure zero-downtime during rollout.

• The ALB/Ingress only routes traffic to pods that pass **readinessProbe** checks — traffic doesn't hit unready pods.

■ Why Rolling Update Over Blue-Green or Canary Here

• Rolling update is resource-efficient — no need to double infrastructure as in blue-green.

• Service was stateless — no session affinity or DB migration complexity that would require more controlled strategies.

• For the **GitOps pipeline project**, Argo CD managed the rollout automatically from Helm chart changes in the Git repo.

■ Deployment Spec (Key Fields)

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
readinessProbe:
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Q6. What is immutable infrastructure in Terraform? How does Terraform support the concept of immutability?

■ What Is Immutable Infrastructure

• Instead of patching or modifying existing servers/resources in-place, you **replace** them entirely with a new version.

• The old resource is destroyed, a new one (with the fix/update) is created. No configuration drift. No 'snowflake servers'.

• Predictable, reproducible, and audit-friendly — the state at any point is fully described by code.

■ How Terraform Enforces Immutability

• **create_before_destroy lifecycle:** Terraform creates the replacement resource first, then destroys the old one — zero-downtime replacements.

• **Forced resource replacement:** Certain attribute changes (e.g., AMI ID, AZ, launch config) force a destroy+create cycle — Terraform will show this explicitly in the plan.

• **Immutable AMIs / Docker images:** Infrastructure is provisioned using versioned, pre-baked AMIs (via Packer) or container images — not SSH'ed into post-launch.

• **No in-place config management:** Terraform doesn't run Ansible/Chef/Puppet post-provision. The resource is fully configured at creation time via user_data or container image.

■ Lifecycle Block Example

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.medium"
```

```
  lifecycle {
    create_before_destroy = true
    ignore_changes        = [tags]
  }
}
```

Q7. Do you maintain a single CI/CD pipeline for all environments (Dev, SIT, UAT, Prod), or separate pipelines? How do you manage environment-specific configurations?

■ Approach Used: Single Pipeline with Environment-Specific Stages

- One Jenkinsfile / GitHub Actions workflow, multiple **stages/jobs** representing each environment.
- Promotion between environments is gate-controlled — manual approval required before SIT → UAT and UAT → Prod.
- The same Docker image (from ECR) is promoted across all environments — **no rebuilding per environment**. This guarantees what's tested in SIT is exactly what goes to Prod.

■ Managing Environment-Specific Configurations

- **Helm values files:** *values-dev.yaml*, *values-sit.yaml*, *values-uat.yaml*, *values-prod.yaml* — environment-specific replicas, resource limits, and endpoints.
- **Kubernetes Namespaces:** Each environment gets its own namespace (dev, sit, uat, prod) for isolation.
- **Secrets via AWS Secrets Manager + External Secrets Operator:** Secrets are pulled at runtime into Kubernetes Secrets — never hardcoded in the pipeline.
- **Jenkins Credentials Store / GitHub Actions Secrets:** AWS credentials and tokens are environment-scoped and never exposed in logs.
- **Argo CD ApplicationSets (GitOps):** In the GitOps project, each environment maps to a separate Argo CD Application pointing to its values file in Git.

■ Helm Deploy Command Per Environment

```
# Dev
helm upgrade --install app ./helm-chart \
  -f values-dev.yaml -n dev

# Prod (with resource + replica overrides)
helm upgrade --install app ./helm-chart \
  -f values-prod.yaml -n prod
```

■ Key Interview Takeaway: Your self-introduction sets the agenda. Only mention a technology if you can explain it with a real scenario from your Hisan Labs internship. Interviewers will drill down on every bullet point on your resume — vague answers on mentioned technologies signal padding.